

Les fonctions récursives

1. Mécanisme d'appel d'une fonction

Une fonction dans un programme informatique est constituée d'un ensemble d'instruction effectuant un traitement particulier à partir de paramètres fournis lors de l'appel de la fonction.

Dans la mémoire, les fonctions et le programme principal sont stockés à des adresses différentes. L'appel de la fonction par le programme principal a pour effet de déplacer le pointeur programme au début de la fonction afin de l'exécuter. Dans le même temps afin d'assurer la continuité du programme au retour de la fonction, les informations suivantes sont mémorisées dans la pile d'appel (call stack) :

- x Les paramètres de la fonction : Pour mémoriser ces valeurs nécessaires à l'exécution de la fonction
- x Une zone réservée pour la valeur de retour
- x L'adresse de retour afin mémoriser l'adresse de l'instruction du programme principal qui sera exécuté au retour de la fonction
- x Éventuellement d'autres informations suivant le langage utilisé (variable local ...)

Lors de l'appel successif de plusieurs fonctions, chaque appel crée un bloc fonction qui sera empilé sur le bloc de la fonction appelante. Ainsi, l'ordre d'appel des fonctions correspond à l'ordre d'empilement des blocs.

Lorsqu'une fonction est terminée, le bloc est dépilé de la pile afin de récupérer les informations nécessaire à la continuité du programme.

Prenons pour exemple le code suivant :

```
def h(x) :  
    return x+1  
def g(x) :  
    return h(x) + 2 + k(x+1)  
def f(x) :  
    return g(x) + 1  
def k(x) :  
    return x+1
```

Q1. Exécuter ce programme sur l'IDE Thonny puis **justifier** le résultat renvoyé par l'appel `f(6)`.

Q2. Relever les adresses des différentes fonctions en mémoire (affichage/heap). **Comparer** l'ordre d'appel des fonctions lors de l'appel `f(5)` (utiliser le mode pas à pas et la visualisation de la pile d'appel).



Mathématiquement cette fonction récursive s'écrit :

$$\begin{aligned}
 n! &= n \times (n-1)! \\
 &\quad \rightarrow = (n-1) \times (n-2)! \\
 &\quad \quad \rightarrow = (n-2) \times (n-3)! \\
 &\quad \quad \quad \rightarrow \dots \\
 &\quad \quad \quad \quad \rightarrow = 2 \times 1! \\
 &\quad \quad \quad \quad \quad \rightarrow = 1
 \end{aligned}$$

Illustration du principe de récursivité
dans la fonction factorielle

Cette fonction peut être implémentée en langage Python de la façon suivante :

```
def factorielle(n):
    assert n >= 0
    if n==1 : return 1
    else : return n*factorielle(n-1)
```

Q4. Justifier que cette fonction est une fonction récursive. **Exécuter** ce programme et **vérifier** le résultat obtenu.

La figure suivante représente l'évolution de la pile d'appel lors de l'appel de la fonction `factorielle(4)`.

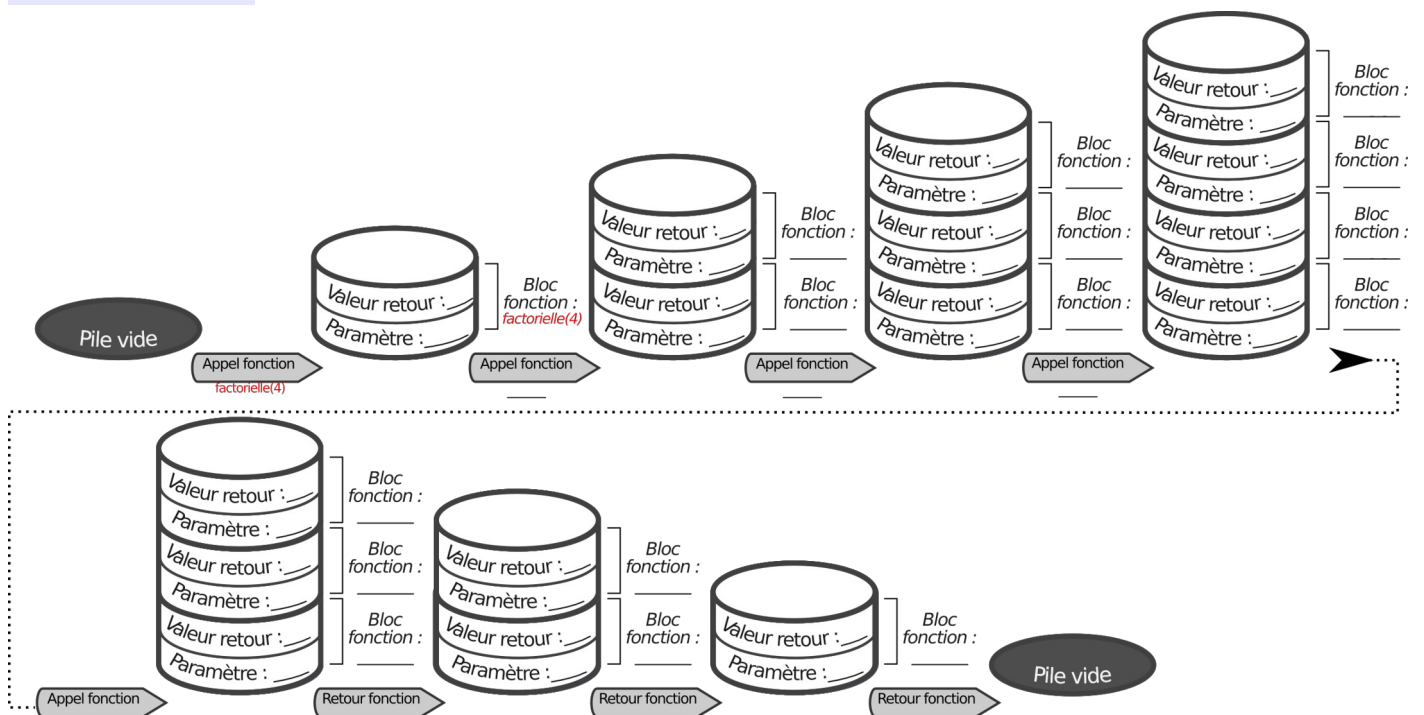


Figure 1: Représentation de la pile d'appel pour `factorielle(4)`

Q5. A partir du programme Python de la fonction factorielle, **compléter** sur la figure 2 l'ordre d'appel et des retours des fonctions. **Compléter** dans chaque bloc la valeur du paramètre transmis ainsi que la valeur de retour au moment où elles apparaissent lors des appels/retour de fonction.



Q6. Vérifier vos résultats en utilisant l'exécution en mode pas à pas et la visualisation de la pile d'appel.

3. Comparaison de la méthode itérative et de la méthode récursives

La même fonction factorielle peut être programmée de manière itérative (avec une boucle for) ou récursive. Les deux programmes sont les suivants :

Version itérative

```
def fact_iterative(n):  
    fact = 1  
    for i in range(1, n+1):  
        fact = fact * i  
    return fact
```

Version récursive

```
def fact_recursive(n):  
    assert n >= 0  
    if n==1:  
        return 1  
    else:  
        return  
        n*fact_recursive(n-1)
```

Q7. Pour les deux méthodes, **déterminer** le nombre de multiplications effectuées pour le cas $n!$. **Comparer** le coût en temps de ces deux algorithmes.

Q8. Exécuter les deux programmes pour $n = 2000$. **Justifier** l'erreur obtenue avec l'algorithme récursif.

Q9. Conclure en comparant le coût en temps et le coût en mémoire de ces deux algorithmes.

